StudyBloom

Claude Code prompt and design reference

Everything Claude Code needs to build the StudyBloom PWA: the full build prompt, plus the design tokens and six screen mockups it should match.

How to use this

1. Create your project folder and a `design` folder inside it.
2. Put `studybloom-design-reference.html` in `design/`. Claude Code reads that file; the pictures in this PDF are the same screens, for you and your teacher.
3. Open a terminal in the project folder and run `claude`.
4. Paste the prompt below (or the .txt version, which is easier to copy). Review the plan Claude Code shows you before saying yes.

The prompt

```
You are building "StudyBloom", a Progressive Web App that helps one student
keep track of school assignments and deadlines. Read this whole brief AND the
design reference file, then show me a plan (folder structure, classes,
database schema, build/deploy steps) and WAIT for my approval before writing
code.

## Design reference (source of truth for the UI)
- Open and study ./design/studybloom-design-reference.html before planning.
  It contains the design tokens (colors, fonts, radius, shadow, spacing) and
  six screen mockups drawn at 360 x 740.
- Match it closely: same colors, fonts, spacing, component shapes, copy,
  and screen layouts. Copy the CSS variables in its :root and .dark blocks
  into the app's stylesheet as the starting point.
- Screens to build, in order, mapped to classes:
  1. Sign in            -> LoginScreen
  2. My tasks (home)    -> TaskListScreen (with the bloom meter)
  3. Add assignment     -> AddTaskScreen (bottom sheet)
  4. Task completed     -> completion state inside TaskListScreen
  5. Plans              -> PricingScreen
  6. Me / settings      -> SettingsScreen (dark mode toggle, install card,
                           demo mode toggle, sign out)
- Follow the notes under each mockup; they describe behavior.
- If something is not covered by the reference, extend it in the same
  style and tell me what you added.

## Problem (from my project brief)
- User: a high school or college student who frequently forgets assignments
  because teachers announce them in class and in different group chats.
- Core job: record assignments and remember their deadlines.
- MVP features: add a task, set a due date, mark a task as completed,
  view unfinished tasks.
```

- 3-minute demo: add an assignment, set its deadline, see it on the task
  list, mark it completed.

Tech stack (strict)
- Language: Java 21, object-oriented programming ONLY. All app logic, models,
  services, and UI rendering must be Java classes. No standalone JS/TS
  frameworks (no React, Vue, etc.).
- Java -> web: compile Java to JavaScript with TeaVM (use its JSO/DOM APIs
  for the UI). Build with Maven.
- Database + auth: Supabase (Postgres + Supabase Auth with email/password).
  Call Supabase's REST API (PostgREST) and Auth endpoints from Java through
  a small TeaVM interop layer. Enable Row Level Security so each student
  only sees their own tasks.
- Hosting: Vercel, as a static site (TeaVM output + index.html + manifest +
  service worker). Include vercel.json and explain the build command and
  output directory.
- Secrets: only the Supabase URL and anon key go in the frontend, loaded
  from env vars at build time. Never put the service role key or Stripe
  secret key in client code. Add a .env.example.

OOP design requirements
Use clear OOP principles, and comment where each one appears
(encapsulation, inheritance, polymorphism, abstraction):
- Models: Task, Student, Subscription (abstract) with subclasses
  FreePlan, ProPlan, MaxPlan that each override methods such as
  getMaxActiveTasks(), canUseReminders(), canUseSubjectTags(),
  canUseSchedule(), canUseNotes().
- Repositories: an interface TaskRepository with SupabaseTaskRepository
  implementing it.
- Services: TaskService, AuthService, SubscriptionService,
  BloomMeterService (counts tasks completed this week for the bloom meter).
- UI: a base abstract Screen class with subclasses LoginScreen,
  TaskListScreen, AddTaskScreen, PricingScreen, SettingsScreen.
  Reusable UI components (TaskCard, BloomMeter, PlanCard, Toast,
  BottomSheet) as their own classes.
- Keep classes small and single-purpose. Private fields with getters/setters.

Subscription plans
- Free (P0): up to 15 unfinished tasks, add/complete/view tasks,
  due-date sorting, overdue highlighting.
- Pro (P49/month): unlimited tasks, subject tags and colors, due-date
  reminders (browser notifications via the service worker).
- Max (P99/month): everything in Pro, plus class schedule and notes per
  subject.
- Show prices with the peso sign (₱) in the UI, as in the Plans mockup.
- Store the plan in a Supabase "subscriptions" table linked to the user.
  Enforce limits in Java (SubscriptionService) AND in the database
  (RLS policy or check) so they can't be bypassed from the browser.
- Payments: use Stripe in TEST MODE with Stripe Checkout / Payment Links.
  Handle the Stripe webhook in one small Supabase Edge Function that
  updates the user's plan. This is the ONLY non-Java code allowed; keep it
  minimal and well commented.
- Also build a "Demo mode" toggle (MockCheckout class) that upgrades the
  plan without real payment, so I can demo Pro/Max in class.
- Locked features show a small "Pro" or "Max" tag that opens PricingScreen.

PWA requirements
- manifest.json with name, short_name, theme_color (#F472B6),
  background_color (#FFF0F5), display: standalone, and icons (192px and
  512px, maskable). Build the icon from the five-petal flower in the

design reference.
- Service worker that caches the app shell so it opens offline and shows cached tasks; queue changes made offline and sync when back online.
- The install card on the Me screen uses the beforeinstallprompt event; on iPhone show "Tap Share, then Add to Home Screen" instead.
- Must pass Lighthouse's PWA installability checks.

Design rules (summary of the reference file)
- Mobile-first, works at 360px wide and scales up to desktop (center the app column, max-width about 480px, on large screens).
- Fonts: Fredoka for titles, Nunito for everything else (Google Fonts, with system fallbacks).
- Light palette: Blush #FFF0F5, Petal #F472B6, Petal soft #FBCFE8, Rose #BE185D, Lavender #E9D5FF, Plum ink #4A1D3F, Overdue #E11D48, Due today #B45309 on #FEF3C7. Dark palette is in the reference file.
- Radius: cards 20px, sheets 28px, inputs 16px, buttons and tags round. Pink-tinted shadows, never grey.
- The bloom meter (five-petal flower, one petal per 20% of this week's tasks completed) is the signature element; keep the rest calm.
- Completing a task: circle fills pink, title struck through, short petal burst (skip it when prefers-reduced-motion is set), bloom meter updates, Undo toast for 5 seconds.
- Tap targets at least 44px, visible focus ring, WCAG AA contrast.
- Support dark mode (follows the system, plus a manual toggle).

Database (Supabase)
Write SQL migration files for: profiles, tasks (id, user_id, title, subject, description, due_date, is_completed, completed_at, created_at), subscriptions (user_id, plan, status, current_period_end), and Max-only tables: schedules, notes. Include RLS policies for each.

Deliverables
1. Plan first, wait for my OK.
2. Working code, built step by step, starting with the MVP (add task, due date, complete, view unfinished), then auth, then PWA, then plans.
3. Unit tests (JUnit) for the services and plan rules.
4. README.md with: setup, Supabase setup steps, env vars, how to run locally, how to deploy to Vercel, how to test Stripe in test mode, how to install on Android and iPhone, and a short section explaining where each OOP principle is used (for my class presentation).
5. After each major step, take a screenshot or describe how the screen compares with the design reference, and tell me what to test.

Design tokens

Colors, light mode

- **Blush** #FFF0F5 background
- **Petal** #F472B6 primary
- **Petal soft** #FBCFE8 borders
- **Rose** #BE185D headings
- **Lavender** #E9D5FF accent
- **Plum ink** #4A1D3F text
- **Overdue** #E11D48
- **Due today** #B45309 on #FEF3C7

Colors, dark mode

- **Night plum** #2A1024 background
- **Card** #3A1631
- **Petal** #F472B6 primary
- **Rose light** #F9A8D4 headings
- **Ink** #FCE7F3 text
- **Overdue** #FB7185

Type

Fredoka 600, 26–28px
Screen titles and plan names

Fredoka 500, 15–19px
Group labels, card titles

Nunito 800, 15px
Task titles, buttons (16px)

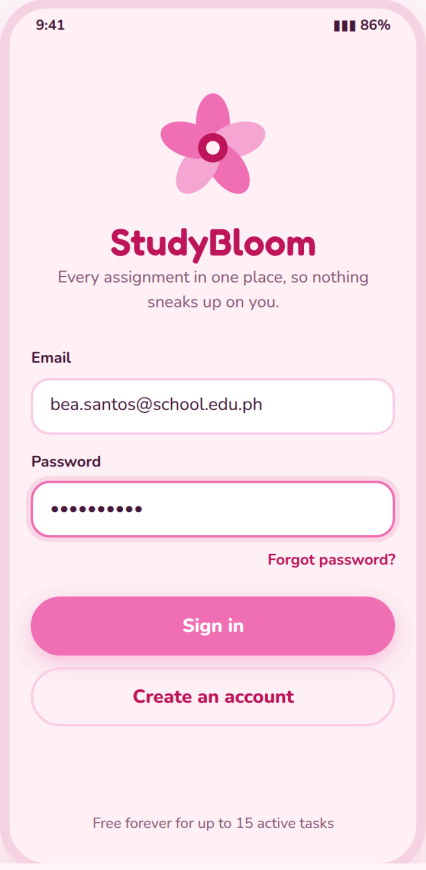
Nunito 400–700, 12.5–15px
Body, due dates, tags

Shape and rules

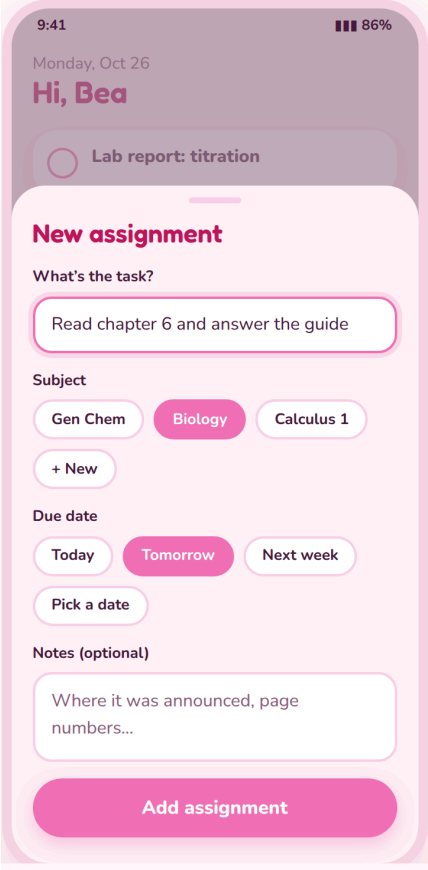
- Radius: cards 20px, sheets 28px, inputs 16px, buttons and tags fully round.
- Shadow: 0 6px 18px rgba(190,24,93,0.10), pink tinted, never grey.
- Spacing: 18px screen padding, 10px between task cards, 14px between fields.
- Tap targets at least 44px. Visible pink focus ring on every control.
- The bloom meter is the one signature element. Keep everything else calm.
- Respect reduced motion: skip the petal burst, keep the toast.

Fonts: Fredoka (titles) and Nunito (body), both free on Google Fonts. Signature element: the five-petal bloom meter on the home screen.

Screens 1–3







1. Sign in

LoginScreen. Supabase email and password. The focused field gets a petal border and soft glow. Errors appear under the field in overdue red and say how to fix it.

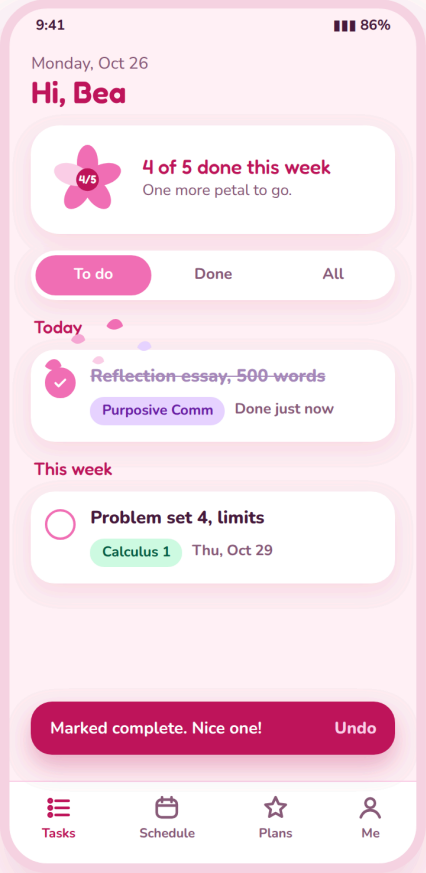
2. My tasks (home)

TaskListScreen. Unfinished tasks grouped into Overdue, Today, This week, Later, sorted by due date. The bloom meter fills one petal per task completed this week (5 petals, each petal = 20% of the week's tasks). Tap the circle to complete.

3. Add assignment

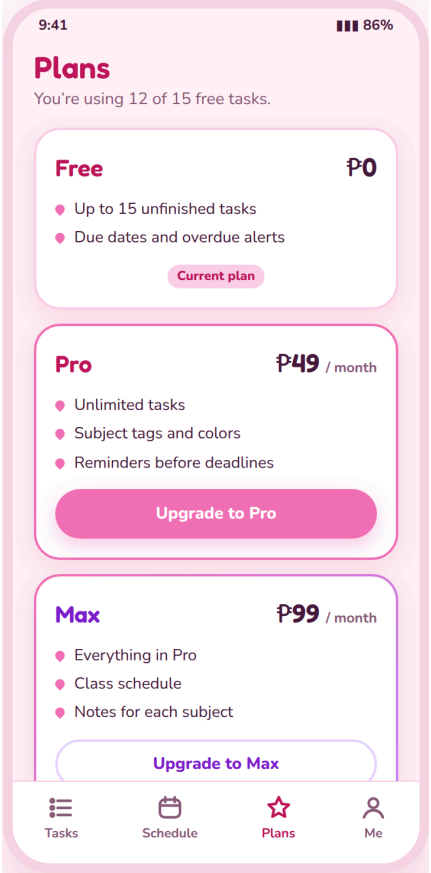
AddTaskScreen as a bottom sheet over the list. Only the title and due date are required. Quick date chips first, full date picker under "Pick a date". Subject tags are a Pro feature; on Free the Subject row is hidden.

Screens 4–6



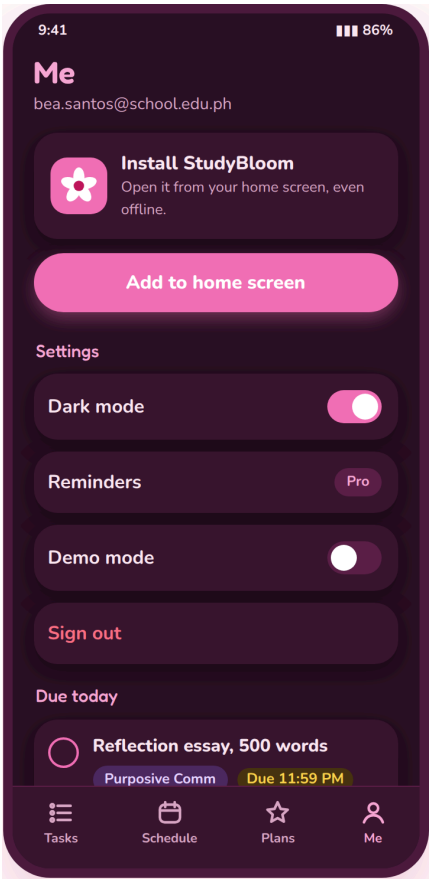
4. Task completed

The one celebratory moment. Circle fills pink, title gets struck through, five petals drift up for about 1 second, the bloom meter gains a petal, and an Undo toast stays for 5 seconds before the card moves to Done.



5. Plans

PricingScreen. Shows usage against the Free limit. Current plan is outlined softly, the recommended upgrade gets the petal border, Max gets a pink-to-lavender border. Buttons open Stripe Checkout, or MockCheckout in demo mode.



6. Me, settings and install (dark mode)

SettingsScreen shown in dark mode. The install card appears only when the browser fires beforeinstallprompt; on iPhone it shows “Tap Share, then Add to Home Screen” instead. Locked features show a Pro or Max tag that opens Plans.